Hive Project Report ADSAI

Aditya Arsh
SR No. 23812
adityaarsh@iisc.ac.in

Bhadbhade Anuj Sujit
SR No. 23904
anujsujit@iisc.ac.in

Jithendra Rao Kasibhatla
SR No. 23646
jithendrarao@iisc.ac.in

Muralidhar Rao M
SR No. 23436
muralidharr@iisc.ac.in

Saksham Agrawal
SR No. 23891
sakshama@iisc.ac.in

Shivey Ravi Guttal
SR No. 23915
shiveyravi@iisc.ac.in

Vijay Bharathram Amaruvi
SR No. 23623
vijayba@iisc.ac.in

Abstract

In this project, we built a second brain system that ingests conversations involving multiple people and performs structured extraction and embedding-based pre-processing and stores them in a query-optimized format that works well with MCP. We implemented this using a combination of LLM models, vector embeddings, and LangGraph for orchestrating tool calling, and packaged everything into a dockerized application complete with a frontend.

1 Introduction

The problem of extracting multi-person conversations is fairly non-trivial since we need to not only isolate the voices but we also need to identify the people and also identify unkown voices. Additionally, simply storing the plain text or doing a naive keyword search is not enough to meaningfully query the data. Hence, we buily a pipeline which firstly ingests the audio and does the diarization and identification, this script is then pre-processed and the relational database, vector database and knowledge graph is populated. This is then accessible via MCP which uses these to answer user questions with high accuracy. We discuss each of these parts in more detail in the following sections.

2 Speaker Recognition and Diarization

The first step in the pipeline consists of taking unprocessed audio and passing it through AssemblyAI [1] for speaker recognition, which provides timestamped audio parts grouped by the speaker. Then, we create speaker embeddings by feeding the audio through a SpeechBrain model [2]. With each segmented audio, the audio slice corresponding to it is first taken out, then normalized, resampled, and finally, embedded. The embeddings at the segment level are merged into a weighted mean such that each speaker cluster gets one average embedding.

Then, each cluster embedding is tested against the known speaker embeddings using cosine similarity. If the similarity is above the defined threshold, the cluster is allocated to the respective known speaker, otherwise, it is given an unkown label. The pipeline finally delivers a structured transcript that encompasses the timestamps, deduced speaker identities, and text data. This is then utilized to populate our databases.

3 Relational Database and Vector Database

This is the schema we used for our Relational Database. The design has entities, actions, events, dates, and locations fields separated into their respective tables. The many-to-many linking tables (`entity_events` and `entity_actions`) capture richer relationships such as multiple persons attending the same event or a single person being involved in several actions, etc. The actual conversations themselves are recorded in the lines table so that fine details are not lost. And the linking is done appropriately such that they are easily queriable.

In addition to the relational structure, summaries are kept with vector embeddings, and a linking table maps each summary back to the exact lines it abstracts. The embedding model we used is Gemini's model [3]. This combination of normalized relational data and vector representations is able to maintain exact grounding while allowing flexible semantic search and retrieval. We use NeonDB [4] to manage our database since it is easy to manage and stays persistent.

4 Knowledge Graph

This is the schema we used for our knowledge graph. The graph captures conceptual entities such as Person, Topic, Emotion, Item, and Goal, with typed edges representing relationships like `HAS_OPINION`, `FEELS, LIKES, KNOWS`, and `HAD_INTENT`. These directed relations allow us to model higher-level semantic links that are difficult to express purely through relational schemas.

We use Neo4j [5] as our graph database because its native property-graph model and Cypher query language make it straightforward to encode, traverse, and query these rich connections.

5 LangGraph

6 Docker and FastAPI

7 MCP

A Model Context Protocol is designed to query and retrieval from the databases. The MCP server is connected to Gemini-2.5 Flash. The tools exposed to the MCP for retrieval are as

8 LLM Evaluations

LLMs were evaluated on their ability to extract information from the transcript after processing by speechbrain. 10 models were tested, chosen on their size and accessibility to us. The LLMs were evaluated based on several metrics, including adherence to json schema, similarity to a golden dataset, and scored by a larger model. We found that Gemini 2.5 Flash, closely followed by Openai OSS (20B) were the best models for this task.

References

[1] AssemblyAI. *AssemblyAI Speech-to-Text and Audio Intelligence API*. 2025. URL: `https://www.assemblyai.com/`.

[2] Mirco Ravanelli et al. *SpeechBrain: A General-Purpose Speech Toolkit*. 2021. arXiv: `2106.04624 [eess.AS]`. URL: `https://arxiv.org/abs/2106.04624`.

[3] Google DeepMind. *Google Gemini Text Embedding Model: text-embedding-004*. `https://ai.google.dev/models/embedding/text-embedding-004`. 2024.

[4] Neon Technologies, Inc. *Neon Database: Serverless Postgres with Separation of Storage and Compute*. `https://neon.tech/`. Accessed: 2025-02-17. 2025.

[5] Neo4j, Inc. *Neo4j Graph Database: Native Graph Storage and Querying*. `https://neo4j.com/`. Accessed: 2025-02-17. 2025.

A Relevant Images

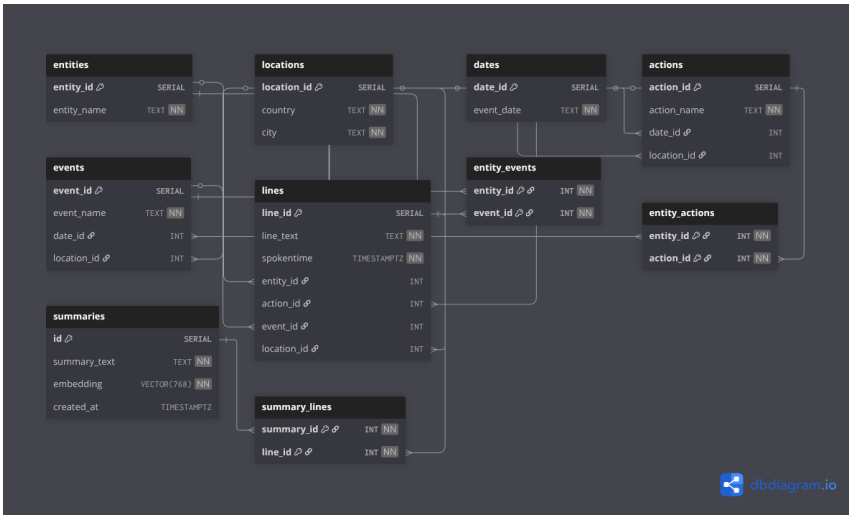


Figure 1: RDB Schema

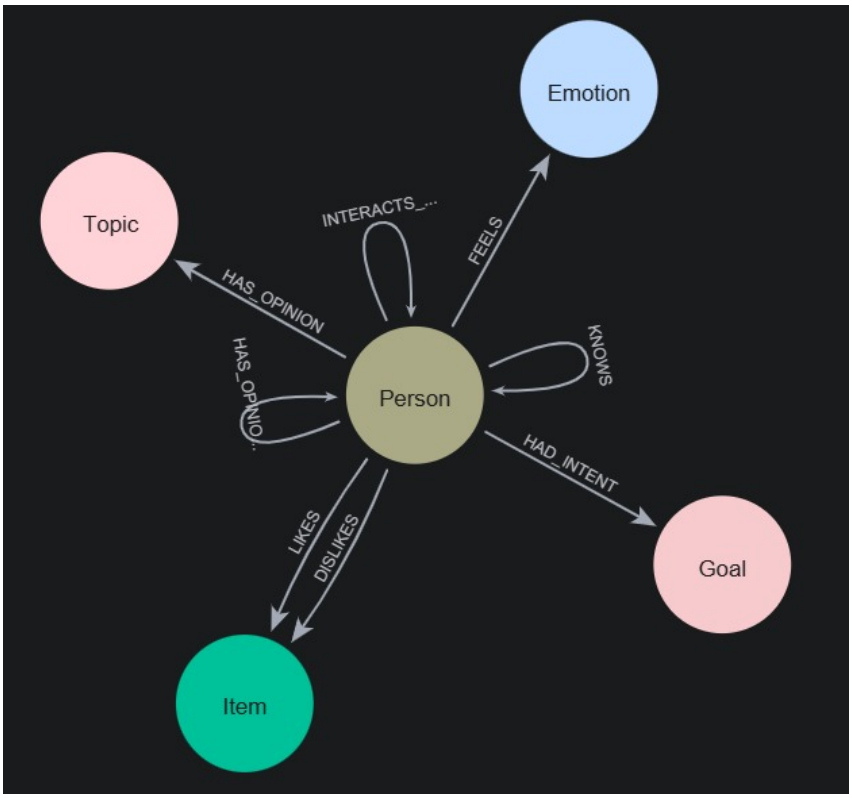


Figure 2: Knowledge Graph Schema

B LLM Evals

B.1 The Models chosen for the task

The following models were chosen for the task: Gemma 4B, Gemma 1B, Gemini 2.5 Flash, LlaMa 3.2 8B, LlaMa 3.3 70B, Mistral 7B, Mistral 24B, OpenAI OSS 20B, Deepseek R1 8B, Hermes 7B. This represents a mixture of models of varying sizes, to explore the possibility of running this locally using minimal hardware, and comparing the results as parameter size increases

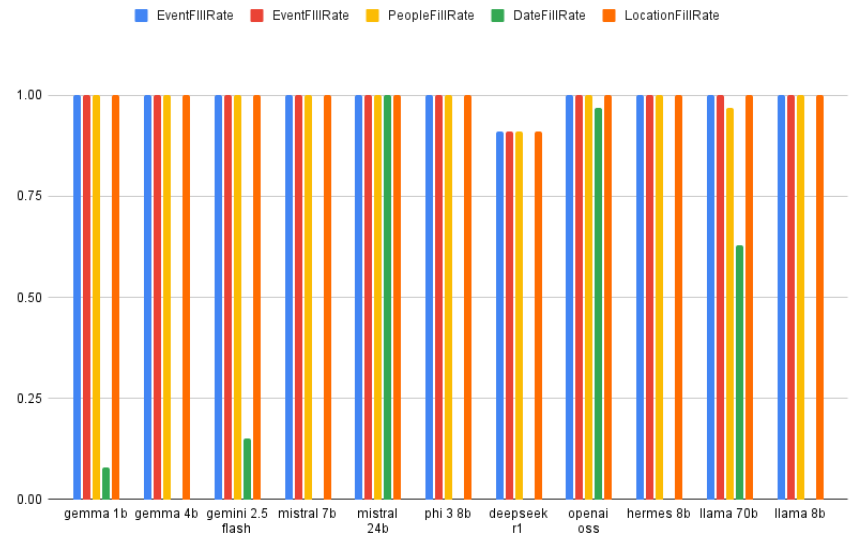
B.2 Testing Methodology

Firstly, the models were tested on their ability to fill the fields and adhere to the JSON structure.

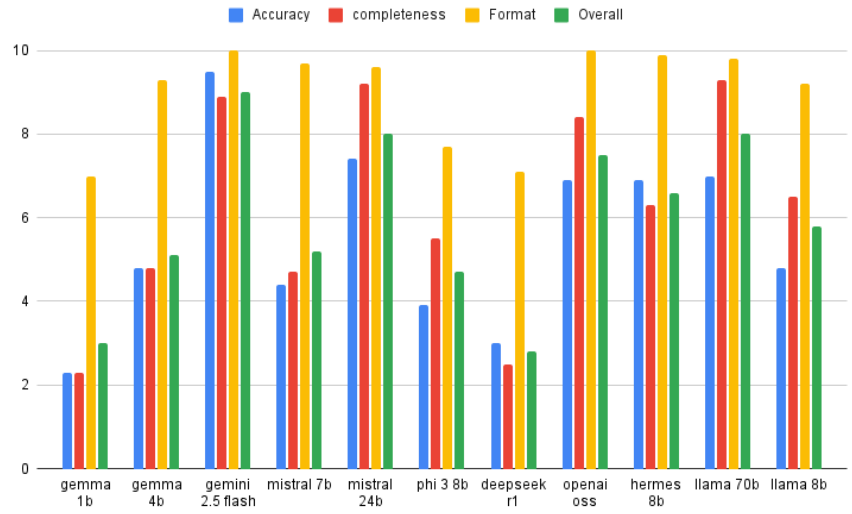
A golden dataset was prepared, consisting of conversation snippets containing events and actions, and "filler" content. The results were then compared to handcrafted JSON outputs for the golden dataset. F1 score, precision, recall were calculated, using semantic similarity and a threshold of 0.8.

The models were then graded on their output by Gemini 2.5 pro on a larger dataset on their accuracy, format, completeness and overall score, and averaged.

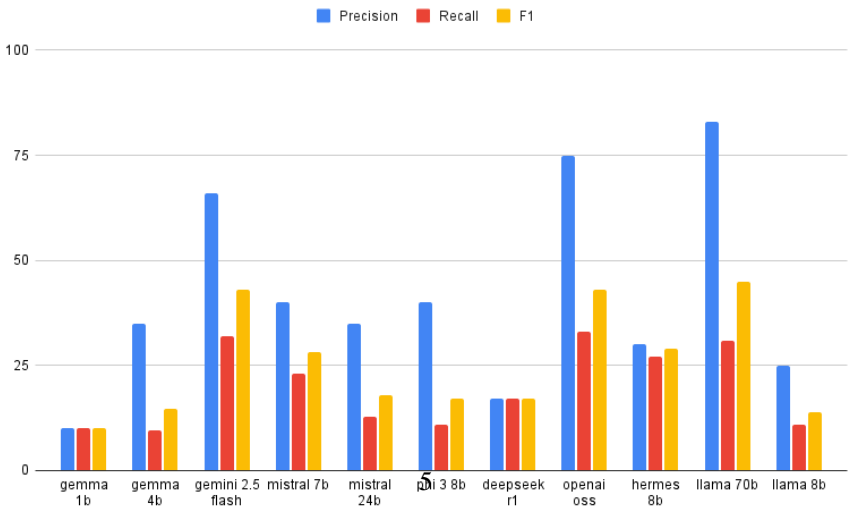
B.3 Results



(a) Field-Filling Rate



(b) Gemini 2.5 Pro grade



(c) Precision, Recall, F1